รายงานโครงงาน

เรื่อง การทำนายเกี่ยวกับคอมพิวเตอร์

เสนอ

ดร. จิรายุ เพชรแหน

จัดทำโดย

66010385 นายธีร์ธวัช ศรีบุญ

66010695 นายรัชชานนท์ คล้ายสุบรรณ์

รายงานฉบับนี้เป็นส่วนหนึ่งของวิชา INTRODUCTION TO MACHINE LEARNING AND DEEP LEARNING

รหัสวิชา 01076641 กลุ่มเรียน 1

ภาคเรียนที่ 2 ปีการศึกษา 2568

ที่มาของโครงงาน

โครงการนี้มีวัตถุประสงค์เพื่อวิเคราะห์และคาดการณ์ราคาคอมพิวเตอร์ รวมถึงประเมินความคุ้มค่าในการตัดสินใจซื้อคอมพิวเตอร์โดยใช้เทคนิคทาง Machine Learning โดยอาศัยชุดข้อมูลที่ประกอบด้วยคุณลักษณะสำคัญของเครื่องคอมพิวเตอร์ เรามุ่งเน้นการค้นหารูปแบบและความสัมพันธ์ระหว่างสเปกของเครื่องกับราคา เพื่อสร้างแบบจำลองเชิงทำนายที่สามารถประมาณราคาที่เหมาะสมของคอมพิวเตอร์แต่ละเครื่องได้อย่างแม่นยำ อีกทั้งยังวิเคราะห์ความคุ้มค่าโดยเปรียบเทียบประสิทธิภาพต่อราคาที่จ่าย เพื่อช่วยให้ผู้บริโภคสามารถตัดสินใจเลือกซื้อคอมพิวเตอร์ได้อย่างมีข้อมูลรองรับ ลดความเสี่ยงในการจ่ายเกินความจำเป็น และเพิ่มประสิทธิภาพในการลงทุนด้านอุปกรณ์เทคโนโลยีในยุคดิจิทัลปัจจุบัน

เป้าหมายของโครงงาน

การวิเคราะห์ข้อมูลเชิงสำรวจ (Exploratory Data Analysis หรือ EDA)

ทำการสำรวจและทำความเข้าใจลักษณะของข้อมูลเบื้องต้น วิเคราะห์การกระจายตัวของข้อมูลในแต่ละตัวแปร พร้อมทั้งแสดงภาพจำนวนค่าของข้อมูลในแต่ละคอลัมน์ เพื่อค้นหารูปแบบ ความสัมพันธ์ และความผิดปกติของข้อมูล

การสร้างและการคัดเลือกคุณลักษณะ

ปรับปรุงและเตรียมคุณลักษณะให้เหมาะสมต่อการนำไปใช้กับโมเดล เช่น การเข้ารหัสตัวแปรที่ไม่ใช่ตัวเลข และการคัดเลือกตัวแปรที่มีอิทธิพลต่อราคามากที่สุด

การประมวลผลข้อมูลเบื้องต้น

จัดการข้อมูลให้อยู่ในรูปแบบที่เหมาะสมต่อการทำนาย เช่น การจัดกลุ่มราคาให้อยู่ในช่วงละ 25 หน่วย เพื่อใช้ในการทำนายและลดความซับซ้อนของข้อมูล

การพัฒนาและการประเมินโมเดล

1. นำโมเดล Regression หลายแบบมาใช้เพื่อวิเคราะห์วามสัมพันธ์ตัวแปรต้นต่อตัวแปรตาม ได้แก่:
    a. Linear regression
    b. Multiple regression
    c. Polynomial regression
    d. A better and extracurricular classification model (mandatory)
2. นำโมเดล Classification หลายแบบมาเพื่อจำแนก ได้แก่
    a. Logistic regression
    b. Decision tree
    c. Ensemble learning: Random Forest
    d. Support vector machine
    e. Dimensionality reduction for random forest/support vector machine
    f. Unsupervised k-mean and agglomerative clustering
    g. Perceptron & Single-layer perceptron (SLP)
    h. Multi-layer perceptron (MLP)

i. A better and extracurricular classification model (mandatory)

3. ประเมินโมเดลโดยใช้เกณฑ์วัดต่างๆ เช่น ค่าการสูญเสีย (Loss), ค่า Mean Squared Error (MSE) และค่า Coefficient of Determination (R^2)

เลือกและการประยุกต์ใช้โมเดลสุดท้าย

ดำเนินการคัดเลือกโมเดลที่มีประสิทธิภาพดีที่สุดจากผลการประเมิน โดยพิจารณาจากค่าเกณฑ์วัดต่าง ๆ เช่น Loss, MSE และ R^2 ร่วมกับความสามารถของโมเดลในการอธิบายความสัมพันธ์ของข้อมูล (Interpretability) และความเหมาะสมต่อการใช้งานจริง

ข้อมูลจำเพาะของข้อมูล (Dataset Specification)

ชุดข้อมูล (Dataset) ที่ใช้ในโครงงานนี้ประกอบด้วยคุณลักษณะหลากหลายรายการ ซึ่งสะท้อนรายละเอียดด้านสเปกและคุณสมบัติของคอมพิวเตอร์แต่ละเครื่อง โดยข้อมูลครอบคลุมทั้งตัวแปรเชิงตัวเลข (Numerical Features) และตัวแปรเชิงหมวดหมู่ (Categorical Features) ซึ่งล้วนมีบทบาทสำคัญต่อการวิเคราะห์และการคาดการณ์ราคา โดยมีรายละเอียดของชุดข้อมูลดังกล่าวต่อไปนี้

1. device_type
   - ประเภท: ตัวแปรเชิงหมวดหมู่
   - คำอธิบาย: ประเภทอุปกรณ์
   - ค่าที่เป็นไปได้ เช่น Labtop, Desktop
2. branch
   - ประเภท: ตัวแปรเชิงหมวดหมู่
   - คำอธิบาย: แบรนด์ของคอมพิวเตอร์
   - ค่าที่เป็นไปได้ เช่น Lenovo, HP และค่าอื่นๆ
3. model
   - ประเภท: ตัวแปรเชิงหมวดหมู่
   - คำอธิบาย: รุ่นของคอมพิวเตอร์
   - ค่าที่เป็นไปได้ เช่น Samsung Forge XDI, Lenovo Strix BIE และค่าอื่นๆ
4. release_year
   - ประเภท: ตัวแปรเชิงปริมาณ
   - คำอธิบาย: ปีที่วางจำหน่ายคอมพิวเตอร์
   - ค่าที่เป็นไปได้ เช่น 2018 - 2025
5. os
   - ประเภท: ตัวแปรเชิงหมวดหมู่
   - คำอธิบาย: ระบบปฏิบัติการ
   - ค่าที่เป็นไปได้ เช่น Windows, macOS และค่าอื่นๆ
6. form_factor
   - ประเภท: ตัวแปรเชิงหมวดหมู่
   - คำอธิบาย: กลุ่มการใช้งาน
   - ค่าที่เป็นไปได้ เช่น Mainstream, Gaming และค่าอื่นๆ

7. cpu_branch
    - ประเภท: ตัวแปรเชิงหมวดหมู่
    - คำอธิบาย: แบรนด์ของ CPU
    - ค่าที่เป็นไปได้ เช่น Intel, AMD และค่าอื่นๆ
8. cpu_model
    - ประเภท: ตัวแปรเชิงหมวดหมู่
    - คำอธิบาย: รุ่นของ CPU
    - ค่าที่เป็นไปได้ เช่น Apple M2 Pro, Intel i5-11129 และค่าอื่นๆ
9. cpu_tier
    - ประเภท: ตัวแปรเชิงปริมาณ
    - คำอธิบาย: ระดับของ CPU
    - ค่าที่เป็นไปได้ เช่น 1 - 6
10. cpu_cores
    - ประเภท: ตัวแปรเชิงปริมาณ
    - คำอธิบาย: จำนวนคอร์ใน CPU
    - ค่าที่เป็นไปได้ เช่น 4 - 28
11. cpu_threads
    - ประเภท: ตัวแปรเชิงปริมาณ
    - คำอธิบาย: จำนวนเธรดใน CPU
    - ค่าที่เป็นไปได้ เช่น 4 - 56
12. cpu_base_ghz
    - ประเภท: ตัวแปรเชิงปริมาณ
    - คำอธิบาย: ความเร็วสัญญาณนาฬิกาของ CPU
    - ค่าที่เป็นไปได้ เช่น 2 - 3.4
13. cpu_boost_ghz
    - ประเภท: ตัวแปรเชิงปริมาณ
    - คำอธิบาย: ความเร็วโอเวอร์คล็อก CPU
    - ค่าที่เป็นไปได้ เช่น 2.8 - 4.5
14. gpu_branch
    - ประเภท: ตัวแปรเชิงหมวดหมู่
    - คำอธิบาย: แบรนด์ของ GPU
    - ค่าที่เป็นไปได้ เช่น Nvidia, Apple และค่าอื่นๆ
15. gpu_model
    - ประเภท: ตัวแปรเชิงหมวดหมู่
    - คำอธิบาย: รุ่นของ GPU
    - ค่าที่เป็นไปได้ เช่น Apple Integrated, RTX 40 70 และค่าอื่นๆ

16. gpu_tier
    - ประเภท: ตัวแปรเชิงปริมาณ
    - คำอธิบาย: ระดับของ GPU
    - ค่าที่เป็นไปได้ เช่น 1 - 6
17. vram_gb
    - ประเภท: ตัวแปรเชิงปริมาณ
    - คำอธิบาย: VRAM ใน GPU
    - ค่าที่เป็นไปได้ เช่น 0 - 16
18. ram_gb
    - ประเภท: ตัวแปรเชิงปริมาณ
    - คำอธิบาย: แรมในคอมพิวเตอร์
    - ค่าที่เป็นไปได้ เช่น 8 - 144
19. storage_type
    - ประเภท: ตัวแปรเชิงหมวดหมู่
    - คำอธิบาย: ประเภทของที่เก็บข้อมูล
    - ค่าที่เป็นไปได้ เช่น NVMe, SSD และค่าอื่นๆ
20. storage_gb
    - ประเภท: ตัวแปรเชิงปริมาณ
    - คำอธิบาย: GB ในพื้นที่เก็บข้อมูล
    - ค่าที่เป็นไปได้ เช่น 256 - 4096
21. storage_drive_count
    - ประเภท: ตัวแปรเชิงปริมาณ
    - คำอธิบาย: จำนวนไดรฟ์จัดเก็บข้อมูล
    - ค่าที่เป็นไปได้ เช่น 1 - 4
22. display_type
    - ประเภท: ตัวแปรเชิงหมวดหมู่
    - คำอธิบาย: ประเภทหน้าจอ
    - ค่าที่เป็นไปได้ เช่น LED, OLED และค่าอื่นๆ
23. display_size_in
    - ประเภท: ตัวแปรเชิงปริมาณ
    - คำอธิบาย: ขนาดหน้าจอแสดงผล
    - ค่าที่เป็นไปได้ เช่น 13.3 - 34
24. resolution
    - ประเภท: ตัวแปรเชิงหมวดหมู่
    - คำอธิบาย: ความละเอียดหน้าจอ
    - ค่าที่เป็นไปได้ เช่น 1920x1080, 2560x1600 และค่าอื่นๆ

25. refresh_hz
    - ประเภท: ตัวแปรเชิงปริมาณ
    - คำอธิบาย: อัตราการรีเฟรชในหน่วยเฮิรตซ์
    - ค่าที่เป็นไปได้ เช่น 60 - 240
26. battery_wh
    - ประเภท: ตัวแปรเชิงปริมาณ
    - คำอธิบาย: แบตเตอรี่ในหน่วยวัตต์-ชั่วโมง
    - ค่าที่เป็นไปได้ เช่น 0 - 99
27. charger_watts
    - ประเภท: ตัวแปรเชิงปริมาณ
    - คำอธิบาย: เครื่องชาร์จเป็นวัตต์
    - ค่าที่เป็นไปได้ เช่น 0 - 240
28. psu_watts
    - ประเภท: ตัวแปรเชิงปริมาณ
    - คำอธิบาย: แหล่งจ่ายไฟ (หน่วยเป็นวัตต์)
    - ค่าที่เป็นไปได้ 0 - 1200
29. wifi
    - ประเภท: ตัวแปรเชิงหมวดหมู่
    - คำอธิบาย: มาตรฐาน Wi-Fi
    - ความเกี่ยวข้อง:
    - ค่าที่เป็นไปได้ เช่น Wi-Fi 6, Wi-Fi 6E และค่าอื่นๆ
30. bluetooth
    - ประเภท: ตัวแปรเชิงปริมาณ
    - คำอธิบาย: Bluetooth
    - ค่าที่เป็นไปได้ เช่น 4.2 - 5.3
31. weight_kg
    - ประเภท: ตัวแปรเชิงปริมาณ
    - คำอธิบาย: น้ำหนัก (หน่วยเป็นกิโลกรัม)
    - ค่าที่เป็นไปได้ เช่น 0.92 - 16
32. warranty_months
    - ประเภท: ตัวแปรเชิงปริมาณ
    - คำอธิบาย: จำนวนเดือนสำหรับการรับประกัน
    - ค่าที่เป็นไปได้ เช่น 12 - 48
33. price
    - ประเภท: ตัวแปรเชิงปริมาณ
    - คำอธิบาย: Price

- ค่าที่เป็นไปได้ เช่น 373 - 11k

การทำนายเชิงเส้นสำหรับการทำนายประสิทธิภาพการทำงาน (Regression)

ขั้นตอนการเตรียมข้อมูล

```
import kagglehub
from kagglehub import KaggleDatasetAdapter

df = kagglehub.dataset_load(
    KaggleDatasetAdapter.PANDAS,
    "paperxd/all-computer-prices",
    "computer_prices_all.csv"
)

print(df.head())
```

```
Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.13), please consider upgrading to the latest version (1.0.0).
Downloading from https://www.kaggle.com/api/v1/datasets/download/paperxd/all-computer-prices?dataset_version_number=1&file_name=computer_prices_all.csv...
100%|██████████| 17.3M/17.3M [00:01<00:00, 15.4MB/s]
  device_type     brand              model  release_year       os form_factor  \
0     Desktop   Samsung  Samsung Forge XDI          2022  Windows         ATX
1      Laptop   Samsung    Samsung Pro KMB          2022  Windows  Mainstream
2     Desktop    Lenovo   Lenovo Strix BIE          2024    macOS         SFF
3     Desktop      Dell      Dell Cube AXR          2024  Windows         ATX
4      Laptop  Gigabyte   Gigabyte Pro IX1          2024    Linux      Gaming

  cpu_brand         cpu_model  cpu_tier  cpu_cores  ...  resolution  \
0     Intel    Intel i5-11129         3         12  ...   2560x1440
1     Intel    Intel i7-11114         4         12  ...   1920x1080
2       AMD  AMD Ryzen 5 5168         2          8  ...   3440x1440
3       AMD  AMD Ryzen 5 7550         2          6  ...   3440x1440
4       AMD  AMD Ryzen 7 6230         5         16  ...   2560x1600

   refresh_hz  battery_wh charger_watts psu_watts     wifi  bluetooth  \
0          90           0            0       750  Wi-Fi 6        5.1
1          90          56          120         0  Wi-Fi 6        5.3
2         120           0            0       850  Wi-Fi 6        5.0
3         120           0            0       650  Wi-Fi 6        5.2
4          90          80           90         0  Wi-Fi 6        5.2

   weight_kg warranty_months    price
0      11.00             36  1383.99
1       2.03             12  2274.99
2       7.00             24  1879.99
3       6.00             36  1331.99
4       1.50             12  2681.99

[5 rows x 33 columns]
```

```
print("ROWS and COLUMNS:", df.shape)
```

```
ROWS and COLUMNS: (100000, 33)
```

```python
for col in df.columns:
  if df[col].dtype == 'object':
    # df[col] = df[col].str.strip().str.lower()
    print(f"${col}:", df[col].unique())
```

```
$device_type: ['Desktop' 'Laptop']
$brand: ['Samsung' 'Lenovo' 'Dell' 'Gigabyte' 'MSI' 'Apple' 'HP' 'Acer' 'ASUS'
 'Razer']
$model: ['Samsung Forge XDI' 'Samsung Pro KM8' 'Lenovo Strix BIE' ...
 'ASUS Zen LKD' 'ASUS Blade DH6' 'Acer Think L7R']
$os: ['Windows' 'macOS' 'Linux' 'ChromeOS']
$form_factor: ['ATX' 'Mainstream' 'SFF' 'Gaming' 'Micro-ATX' 'Ultrabook' '2-in-1'
 'Mini-ITX' 'Full-Tower' 'Workstation']
$cpu_brand: ['Intel' 'AMD' 'Apple']
$cpu_model: ['Intel i5-11129' 'Intel i7-11114' 'AMD Ryzen 5 5168' ... 'Intel i3-14276'
 'AMD Ryzen 3 6943' 'AMD Ryzen 9 4438']
$gpu_brand: ['NVIDIA' 'AMD' 'Apple' 'Intel']
$gpu_model: ['RTX 40 60' 'RTX 40 80' 'RTX 40 50' 'RX 7000 60' 'RTX 30 80 Ti'
 'RTX 40 90' 'Apple Integrated' 'RTX 30 50' 'RTX 30 60' 'RX 6000 80'
 'RTX 30 70' 'RTX 40 80 Ti' 'RTX 40 70' 'RX 7000 80 XT' 'RX 5000 50'
 'RX 5000 90' 'RTX 30 80' 'RX 7000 80' 'RTX 30 90' 'RX 6000 60' 'Arc A310'
 'RX 7000 50' 'RX 6000 70' 'RX 6000 90' 'Arc A580' 'Arc B380' 'RX 6000 50'
 'RTX 20 80' 'RX 6000 80 XT' 'RTX 20 70' 'Arc A770' 'RTX 20 60'
 'RTX 20 50' 'Arc A700' 'RX 7000 70' 'Arc A380' 'Arc B700'
 'Arc B770 Limited' 'RX 5000 80' 'RX 7000 90' 'Arc B580' 'RX 5000 60'
 'RTX 20 80 Ti' 'Arc B310' 'Arc B770' 'Arc A770 Limited' 'RX 5000 80 XT'
 'RTX 20 90' 'RX 5000 70']
$storage_type: ['NVMe' 'HDD' 'Hybrid' 'SSD']
$display_type: ['LED' 'OLED' 'IPS' 'Mini-LED' 'VA' 'QLED']
$resolution: ['2560x1440' '1920x1080' '3440x1440' '2560x1600' '3840x2160' '2880x1800']
$wifi: ['Wi-Fi 6' 'Wi-Fi 5' 'Wi-Fi 6E' 'Wi-Fi 7']
```

1. การนำเข้าข้อมูล

   ในขั้นตอนแรกของโครงงาน ได้ทำการนำเข้าชุดข้อมูลจากไฟล์ CSV ชื่อ computer_prices_all.csv สำหรับการวิเคราะห์ข้อมูล เพื่อเตรียมความพร้อมสำหรับกระบวนการทำความสะอาดข้อมูล (Data Cleaning) และการสร้างโมเดลในขั้นตอนถัดไป

2. การตรวจสอบโครงสร้างข้อมูลเบื้องต้น

   หลังจากนำเข้าข้อมูลเรียบร้อยแล้ว ได้ทำการตรวจสอบโครงสร้างและลักษณะทั่วไปของชุดข้อมูล เพื่อทำความเข้าใจองค์ประกอบและความสมบูรณ์ของข้อมูล โดยมีรายละเอียดดังนี้

   a. ขนาดของข้อมูล

      จากการตรวจสอบด้วยคำสั่ง df.shape พบว่าชุดข้อมูลมีขนาด (100000, 33)

      หมายความว่า ประกอบด้วยข้อมูลจำนวน 100,000 แถว (records) และ 33 คอลัมน์ (features) ซึ่งสะท้อนถึงจำนวนตัวอย่างข้อมูลที่เพียงพอต่อการพัฒนาแบบจำลอง Machine Learning

   b. ตัวอย่างข้อมูล 5 แถวแรก

      ใช้คำสั่ง df.head() เพื่อแสดงข้อมูล 5 แถวแรกของชุดข้อมูล ช่วยให้เห็นภาพรวมของโครงสร้างข้อมูล รูปแบบค่าของแต่ละคอลัมน์ และชนิดข้อมูลเบื้องต้น ทำให้สามารถตรวจสอบความถูกต้องของการนำเข้าไฟล์ได้

   c. ค่าคอลัมน์เชิงหมวดหมู่ (Categorical Features Overview)

จากการตรวจสอบค่าเฉพาะ (Unique Values) ของตัวแปรเชิงหมวดหมู่ พบว่าแต่ละคอลัมน์มีจำนวนค่าที่แตกต่างกัน ตั้งแต่ประมาณ 2 ค่า ไปจนถึงประมาณ 50 ค่า ขึ้นอยู่กับลักษณะของตัวแปร เช่น device_type, gpu_model ซึ่งข้อมูลในลักษณะนี้จำเป็นต้องผ่านกระบวนการเข้ารหัส (Encoding) ก่อนนำไปใช้ในการพัฒนาโมเดล

```
import numpy as np
import pandas as pd

gap = 25
labels = np.arange(300, 11001, gap)

first_edge = labels[0] - gap / 2.0
last_edge = labels[-1] + gap / 2.0
bins_edges = np.arange(first_edge, last_edge + gap, gap)

df['predicted_price'] = pd.cut(
    df['price'],
    bins=bins_edges,
    labels=labels,
    right=True,
    include_lowest=True
)

print(df[['price', 'predicted_price']].head(100))
```

```
      price predicted_price
0   1383.99            1375
1   2274.99            2275
2   1879.99            1875
3   1331.99            1325
4   2681.99            2675
..      ...             ...
95  2060.99            2050
96  2564.99            2575
97  1919.99            1925
98  1833.99            1825
99  1418.99            1425

[100 rows x 2 columns]
```

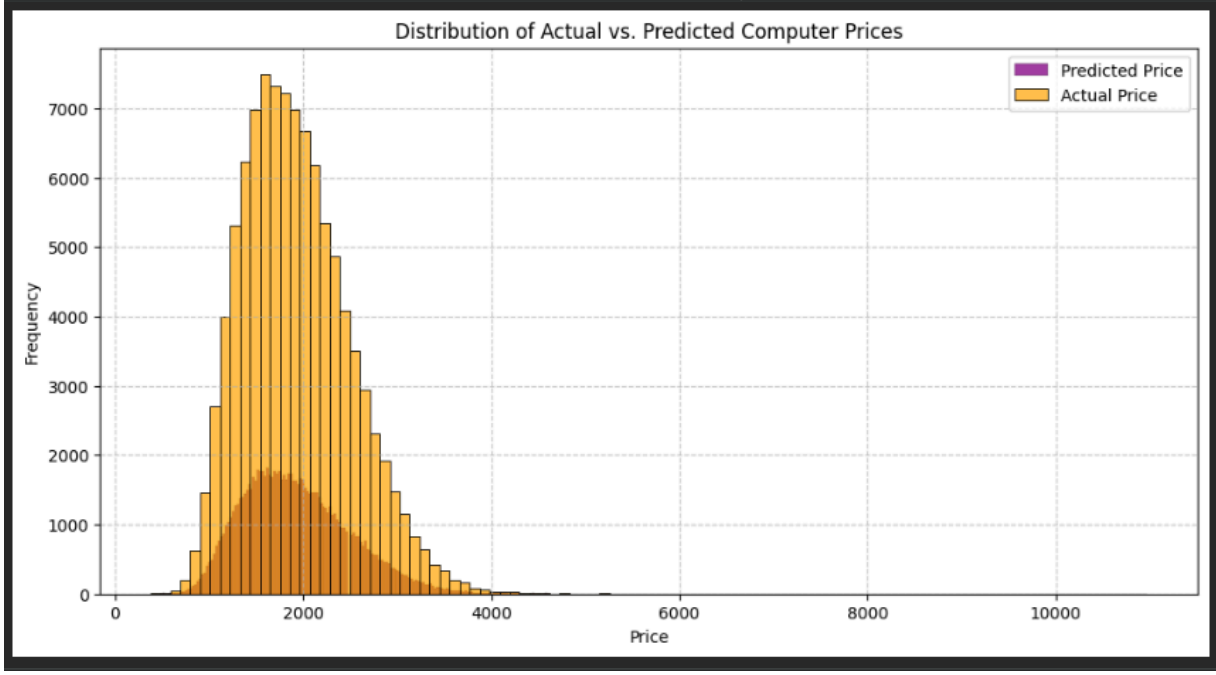
3. การทำให้ราคาอยู่ในช่วง 25 (floor 25)

   ในขั้นตอนการประมวลผลข้อมูล ได้ทำการปรับค่าราคา (price) ให้อยู่ในรูปแบบช่วงละ 25 หน่วย (floor 25) เพื่อลดความละเอียดของข้อมูล เช่น 1,383.99 -> 1,375

การวิเคราะห์ข้อมูลเบื้องต้นด้วยการสร้างภาพข้อมูล (Data Visualization)

```python
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 6))
sns.histplot(df['predicted_price'].astype(float), bins=labels.size, kde=False, color='purple', label='Predicted Price')
sns.histplot(df['price'], bins=100, kde=False, color='orange', alpha=0.7, label='Actual Price')
plt.title('Distribution of Actual vs. Predicted Computer Prices')
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

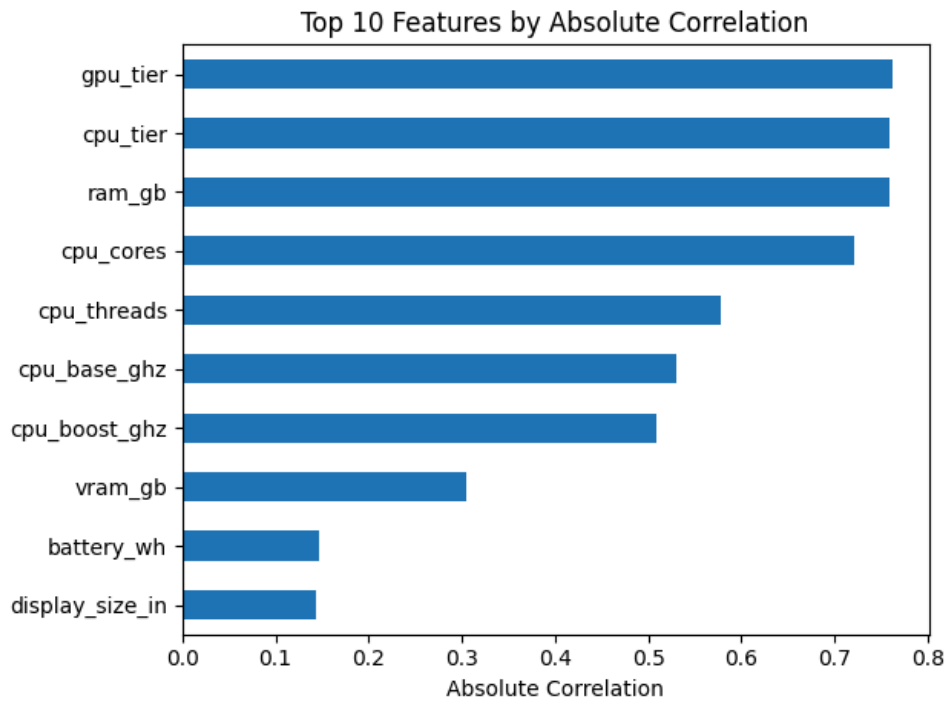


1. กราฟแสดงความสัมพันธ์ระหว่างราคาเดิม (price) กับราคาที่ทำการจัดกลุ่มช่วงละ 25 หน่วย (floor 25)

   การสร้างกราฟนี้มีความสำคัญเนื่องจากช่วยให้เห็นว่า

   a. การทำ Floor 25 ไม่ได้เปลี่ยนแนวโน้มโดยรวมของราคา

   b. การลดความละเอียดของข้อมูลไม่ได้ทำให้โครงสร้างหลักของข้อมูลสูญหาย



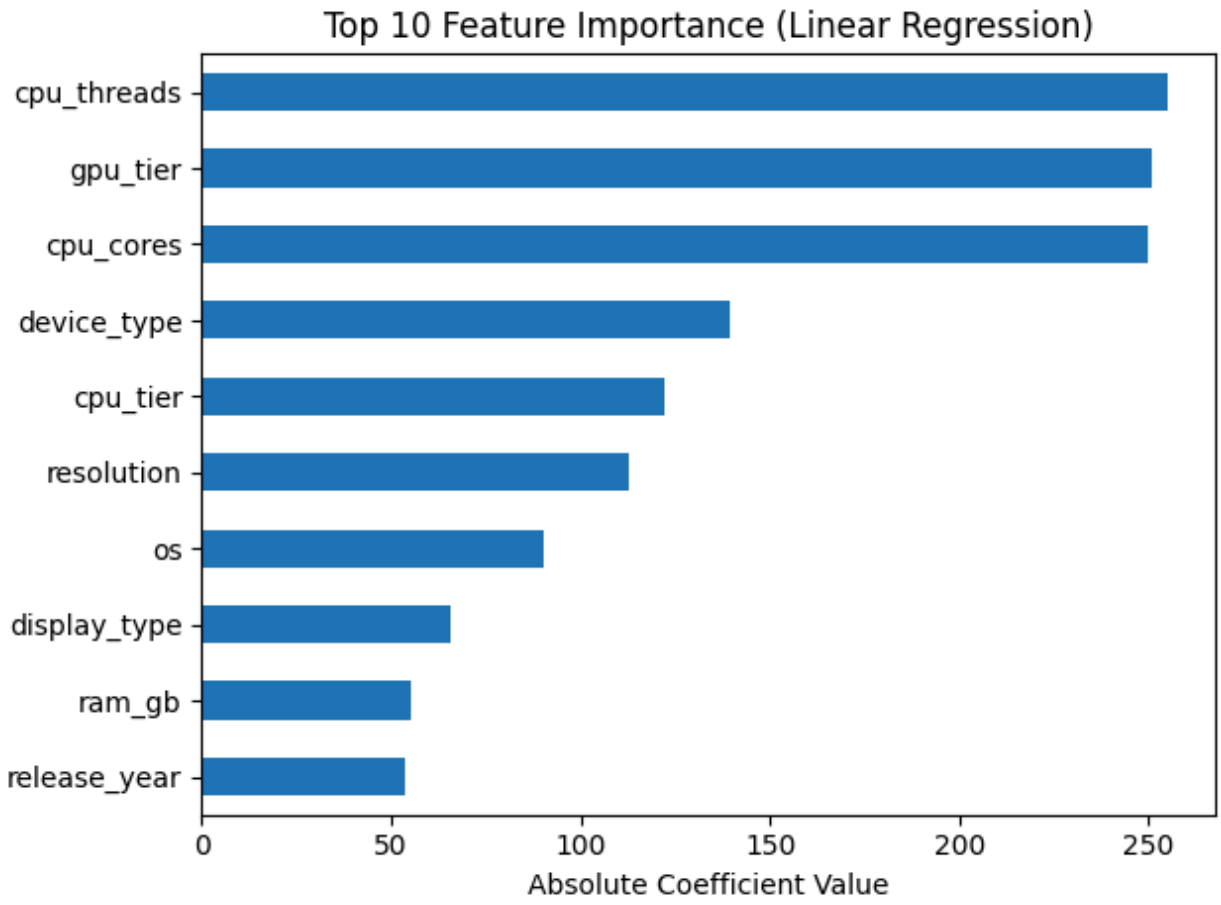
2. กราฟแสดง Top 10 Correlation ระหว่างแต่ละคอลัมน์กับราคาที่ทำ Floor 25

ในขั้นตอนนี้ได้ทำการคำนวณค่าสหสัมพันธ์ (Correlation) ระหว่างตัวแปรทั้งหมดกับตัวแปรราคาในรูปแบบที่ผ่านการจัดกลุ่มช่วงละ 25 หน่วย (floor 25) จากนั้นเลือกเฉพาะ 10 อันดับแรกที่มีค่าสหสัมพันธ์สูงสุด แล้วนำมาแสดงในรูปแบบกราฟแท่ง (Bar Chart) เพื่อให้เห็นความสำคัญของแต่ละปัจจัยอย่างชัดเจน

ผลการวิเคราะห์พบว่าลำดับความสัมพันธ์จากมากไปน้อยมีดังนี้

gpu_tier > cpu_tier > ram_gb > cpu_cores > cpu_threads > cpu_base_ghz > cpu_boost_ghz > vram_gb > battery_wh > display_size_in

การสร้างกราฟนี้มีความสำคัญอย่างยิ่ง เนื่องจากช่วยให้สามารถระบุปัจจัยหลักที่ส่งผลต่อราคาคอมพิวเตอร์ได้อย่างชัดเจน สนับสนุนการคัดเลือกคุณลักษณะ (Feature Selection) สำหรับการสร้างโมเดล



3. กราฟแสดง Top 10 Feature Importance ของ Linear Regression

ในขั้นตอนนี้ได้ทำการวิเคราะห์ความสำคัญของตัวแปร (Feature Importance) จากโมเดล Linear Regression โดยพิจารณาจากค่าสัมประสิทธิ์ (Coefficient) ของแต่ละตัวแปรหลังผ่านกระบวนการปรับสเกลข้อมูล เพื่อเปรียบเทียบอิทธิพลของแต่ละคุณลักษณะที่มีต่อราคาที่ทำการ Floor 25

ผลการวิเคราะห์พบว่าลำดับจากมากไปน้อยมีดังนี้

cpu_threads > gpu_tier > cpu_cores > device_type > cpu_tier > resolution > os > display_type > ram_gb > release_year

การสร้างกราฟ Feature Importance จาก Linear Regression มีความสำคัญ เนื่องจากช่วยให้เข้าใจเชิงลึกว่าโมเดลให้น้ำหนักกับตัวแปรใดมากที่สุด สนับสนุนการอธิบายผลลัพธ์ของโมเดล (Model Interpretability) และช่วยในการตัดสินใจเลือกคุณลักษณะที่เหมาะสมสำหรับการพัฒนาโมเดลในขั้นตอนถัดไป

การฝึกโมเดลเกี่ยวกับ Regression

การเลือกคอลัมน์ที่สำคัญ

จากผลการวิเคราะห์ก่อนหน้า ได้ทำการคัดเลือกเฉพาะคุณลักษณะที่มีความสำคัญสูงเพื่อนำมาใช้ในการฝึกโมเดล โดยเลือกเฉพาะตัวแปรดังต่อไปนี้

gpu_tier, cpu_tier, ram_gb, cpu_cores, cpu_threads, device_type, resolution, os

การเลือกใช้เฉพาะคอลัมน์ที่มีอิทธิพลสูงช่วยลดความซับซ้อนของโมเดล ลดปัญหา Overfitting และเพิ่มประสิทธิภาพในการเรียนรู้ของอัลกอริทึม

ในขั้นตอนการเตรียมข้อมูล ตัวแปรที่ไม่ใช่ตัวเลข ได้แก่ device_type, resolution, และ os ได้ถูกแปลงให้อยู่ในรูปแบบตัวเลขด้วยวิธี Label Encoding เพื่อให้สามารถนำไปใช้กับโมเดล Machine Learning ได้อย่างเหมาะสม

หลังจากกระบวนการเตรียมข้อมูลเสร็จสิ้น ได้ทำการแยกข้อมูลออกเป็น

X (Features) — ประกอบด้วยตัวแปรที่เลือกทั้งหมด
y (Target) — ราคาที่ผ่านการทำ Floor 25
encoder — ตัวแปลงค่าของตัวแปรเชิงหมวดหมู่ที่ถูกเก็บไว้ เพื่อใช้แปลงข้อมูลใหม่ในขั้นตอนการทำนายภายหลัง

กระบวนการนี้ช่วยให้ขั้นตอนการฝึกโมเดลเป็นระบบ มีความชัดเจน และสามารถนำไปประยุกต์ใช้งานจริงได้อย่างถูกต้องและต่อเนื่อง

การเลือกฝึกโมเดล Linear Regression (From Scratch)

1. โครงสร้างของโมเดล

   โมเดลที่สร้างขึ้นประกอบด้วยองค์ประกอบสำคัญ ได้แก่

   - การกำหนดค่า learning rate และจำนวนรอบการเรียนรู้ (n_iterations)
   - การใช้ Gradient Descent ในการอัปเดตค่า coef_ และ intercept_
   - ในแต่ละรอบจะเก็บค่า Loss ไว้ใน loss_history และเก็บค่า Validate ใน val_history
   - ฟังก์ชันสำหรับคำนวณ MSE และ R^2 เพื่อใช้ประเมินโมเดล

2. โหลดข้อมูล

   ทำการเรียกใช้ฟังก์ชัน load_regression_data() เพื่อดึงค่า

   - X (Features)
   - y (Target: price ที่ทำ Floor 25)
   - encoder (สำหรับแปลงข้อมูลเชิงหมวดหมู่)

3. แบ่งข้อมูล Train / Test

   ใช้ train_test_split() แบ่งข้อมูลเป็น

   - 80% สำหรับฝึกโมเดล (Training Set)

- 20% สำหรับทดสอบโมเดล (Test Set)

โดยกำหนด random_state=42 เพื่อให้ได้ผลลัพธ์ที่สามารถทำซ้ำได้

4. ทำ Feature Scaling

   เนื่องจาก Linear Regression ที่ใช้ Gradient Descent มีความไวต่อสเกลของข้อมูล

   จึงทำการ Standardization และเก็บค่า mean และ std ไว้ เพื่อใช้กับข้อมูลใหม่ในอนาคต

5. ฝึกโมเดล

   กำหนดพารามิเตอร์

   - learning_rate = 0.01
   - n_iterations = 3000

   จากนั้นเรียก model.fit() เพื่อเริ่มกระบวนการเรียนรู้

6. การประเมินผลโมเดล

   หลังการฝึก ได้คำนวณค่า

   - Train MSE
   - Test MSE
   - Train R^2
   - Test R^2
   - Training Loss
   - Validation Loss

   เพื่อใช้วัดความแม่นยำของโมเดล และเปรียบเทียบกับโมเดลอื่น

7. การบันทึกโมเดล

   เพื่อให้สามารถนำโมเดลไปใช้งานภายหลังได้ มีการจัดเก็บข้อมูลสำคัญลงในไฟล์ linear_regression_scratch.pkl โดยภายในไฟล์จะประกอบด้วย

   - model (โมเดลที่ฝึกเสร็จแล้ว)
   - mean (ค่าเฉลี่ยสำหรับ scaling)
   - std (ค่าส่วนเบี่ยงเบนมาตรฐานสำหรับ scaling)
   - encoder (ตัวแปลงข้อมูลเชิงหมวดหมู่)

   การจัดเก็บในรูปแบบ pickle ช่วยให้สามารถโหลดโมเดลกลับมาใช้งานสำหรับการทำนาย (Prediction) ได้ทันที โดยไม่ต้องฝึกใหม่

การเลือกฝึกโมเดล Linear Regression (From Scikit-learn)

1. โครงสร้างของโมเดล

   โมเดลประกอบด้วยองค์ประกอบสำคัญ ได้แก่

   - ฟังก์ชัน fit() สำหรับฝึกโมเดล
   - ฟังก์ชัน predict() สำหรับทำนายค่า
   - ฟังก์ชัน mse() สำหรับคำนวณค่า Mean Squared Error
   - ฟังก์ชัน r2_score() สำหรับคำนวณค่า R^2

2. โหลดข้อมูล

ทำการเรียกใช้ฟังก์ชัน load_regression_data() เพื่อดึงค่า

- X (Features)
- y (Target: price ที่ทำ Floor 25)
- encoder (สำหรับแปลงข้อมูลเชิงหมวดหมู่)

3. แบ่งข้อมูล Train / Test

   ใช้ train_test_split() แบ่งข้อมูลเป็น

   - 80% สำหรับฝึกโมเดล (Training Set)
   - 20% สำหรับทดสอบโมเดล (Test Set)

   โดยกำหนด random_state=42 เพื่อให้ได้ผลลัพธ์ที่สามารถทำซ้ำได้

4. ทำ Feature Scaling

   เนื่องจาก Linear Regression ที่ใช้ Gradient Descent มีความไวต่อสเกลของข้อมูล จึงทำการ Standardization และเก็บค่า mean และ std ไว้ เพื่อใช้กับข้อมูลใหม่ในอนาคต

5. ฝึกโมเดล

   สร้างโมเดล model = LinearRegressionSklearn() จากนั้นเรียก model.fit() เพื่อเริ่มกระบวนการเรียนรู้ความสัมพันธ์ระหว่างตัวแปรต้นกับตัวแปรตาม

6. การประเมินผลโมเดล

   หลังการฝึก ได้คำนวณค่า

   - Train MSE
   - Test MSE
   - Train R^2
   - Test R^2

   เพื่อใช้วัดความแม่นยำของโมเดล และเปรียบเทียบกับโมเดลอื่น

7. การบันทึกโมเดล

   เพื่อให้สามารถนำโมเดลไปใช้งานภายหลังได้ มีการจัดเก็บข้อมูลสำคัญลงในไฟล์ linear_regression_sklearn.pkl โดยภายในไฟล์จะประกอบด้วย

   - model (โมเดลที่ฝึกเสร็จแล้ว)
   - mean (ค่าเฉลี่ยสำหรับ scaling)
   - std (ค่าส่วนเบี่ยงเบนมาตรฐานสำหรับ scaling)
   - encoder (ตัวแปลงข้อมูลเชิงหมวดหมู่)

   การจัดเก็บในรูปแบบ pickle ช่วยให้สามารถโหลดโมเดลกลับมาใช้งานสำหรับการทำนาย (Prediction) ได้ทันที โดยไม่ต้องฝึกใหม่

การเลือกฝึกโมเดล Multiple Regression (From Scratch)

1. โครงสร้างของโมเดล

   โมเดลที่สร้างขึ้นประกอบด้วยองค์ประกอบสำคัญ ได้แก่

   - กำหนดค่า learning_rate
   - กำหนดจำนวนรอบการเรียนรู้ n_iterations

- เพิ่ม Bias Term เข้าไปใน X
- ใช้ Mean Squared Error (MSE) เป็น Loss Function
- ในแต่ละรอบเก็บค่า Loss ไว้ใน loss_history และ Validate ใน val_history
- มีฟังก์ชัน mse() และ r2_score() สำหรับประเมินผล

2. โหลดข้อมูล

   ทำการเรียกใช้ฟังก์ชัน load_regression_data() เพื่อดึงค่า

   - X (Features)
   - y (Target: price ที่ทำ Floor 25)
   - encoder (สำหรับแปลงข้อมูลเชิงหมวดหมู่)

3. แบ่งข้อมูล Train / Test

   ใช้ train_test_split() แบ่งข้อมูลเป็น

   - 80% สำหรับฝึกโมเดล (Training Set)
   - 20% สำหรับทดสอบโมเดล (Test Set)

   โดยกำหนด random_state=42 เพื่อให้ได้ผลลัพธ์ที่สามารถทำซ้ำได้

4. ทำ Feature Scaling

   เนื่องจาก Multiple Regression ที่ใช้ Gradient Descent มีความไวต่อสเกลของข้อมูล จึงทำการ Standardization และเก็บค่า mean และ std ไว้ เพื่อใช้กับข้อมูลใหม่ในอนาคต

5. ฝึกโมเดล

   กำหนดพารามิเตอร์

   - learning_rate = 0.01
   - n_iterations = 3000

   จากนั้นเรียก model.fit() เพื่อเริ่มกระบวนการเรียนรู้

6. การประเมินผลโมเดล

   หลังการฝึก ได้คำนวณค่า

   - Train MSE
   - Test MSE
   - Train R^2
   - Test R^2
   - Training Loss
   - Validation Loss

   เพื่อใช้วัดความแม่นยำของโมเดล และเปรียบเทียบกับโมเดลอื่น

7. การบันทึกโมเดล

   เพื่อให้สามารถนำโมเดลไปใช้งานภายหลังได้ มีการจัดเก็บข้อมูลสำคัญลงในไฟล์ multiple_regression_scratch.pkl โดยภายในไฟล์จะประกอบด้วย

   - model (โมเดลที่ฝึกเสร็จแล้ว)
   - mean (ค่าเฉลี่ยสำหรับ scaling)

- std (ค่าส่วนเบี่ยงเบนมาตรฐานสำหรับ scaling)
- encoder (ตัวแปลงข้อมูลเชิงหมวดหมู่)

การจัดเก็บในรูปแบบ pickle ช่วยให้สามารถโหลดโมเดลกลับมาใช้งานสำหรับการทำนาย (Prediction) ได้ทันที โดยไม่ต้องฝึกใหม่

การเลือกฝึกโมเดล Multiple Regression (From Scikit-learn)

1. โครงสร้างของโมเดล

   โมเดลประกอบด้วยองค์ประกอบสำคัญ ได้แก่

   - fit() สำหรับฝึกโมเดล
   - predict() สำหรับทำนายค่า (รองรับทั้งข้อมูล 1 มิติ และหลายมิติ)
   - mse() สำหรับคำนวณค่า Mean Squared Error
   - r2_score() สำหรับคำนวณค่า Coefficient of Determination (R^2)

2. โหลดข้อมูล

   ทำการเรียกใช้ฟังก์ชัน load_regression_data() เพื่อดึงค่า

   - X (Features)
   - y (Target: price ที่ทำ Floor 25)
   - encoder (สำหรับแปลงข้อมูลเชิงหมวดหมู่)

3. แบ่งข้อมูล Train / Test

   ใช้ train_test_split() แบ่งข้อมูลเป็น

   - 80% สำหรับฝึกโมเดล (Training Set)
   - 20% สำหรับทดสอบโมเดล (Test Set)

   โดยกำหนด random_state=42 เพื่อให้ได้ผลลัพธ์ที่สามารถทำซ้ำได้

4. ทำ Feature Scaling

   เนื่องจาก Multiple Regression ที่ใช้ Gradient Descent มีความไวต่อสเกลของข้อมูล จึงทำการ Standardization และเก็บค่า mean และ std ไว้ เพื่อใช้กับข้อมูลใหม่ในอนาคต

5. ฝึกโมเดล

   สร้างโมเดล model = MultipleRegressionSklearn() จากนั้นเรียก model.fit() เพื่อเริ่มกระบวนการเรียนรู้ความสัมพันธ์ระหว่างตัวแปรต้นกับตัวแปรตาม

6. การประเมินผลโมเดล

   หลังการฝึก ได้คำนวณค่า

   - Train MSE
   - Test MSE
   - Train R^2
   - Test R^2

   เพื่อใช้วัดความแม่นยำของโมเดล และเปรียบเทียบกับโมเดลอื่น

7. การบันทึกโมเดล

เพื่อให้สามารถนำโมเดลไปใช้งานภายหลังได้ มีการจัดเก็บข้อมูลสำคัญลงในไฟล์ multiple_regression_sklearn.pkl โดยภายในไฟล์จะประกอบด้วย

- model (โมเดลที่ฝึกเสร็จแล้ว)
- mean (ค่าเฉลี่ยสำหรับ scaling)
- std (ค่าส่วนเบี่ยงเบนมาตรฐานสำหรับ scaling)
- encoder (ตัวแปลงข้อมูลเชิงหมวดหมู่)

การจัดเก็บในรูปแบบ pickle ช่วยให้สามารถโหลดโมเดลกลับมาใช้งานสำหรับการทำนาย (Prediction) ได้ทันที โดยไม่ต้องฝึกใหม่

การเลือกฝึกโมเดล Polynomial Regression (From Scratch)

1. โครงสร้างของโมเดล

   โมเดลที่สร้างขึ้นประกอบด้วยองค์ประกอบสำคัญ ได้แก่

   - กำหนดพารามิเตอร์หลัก
     - degree : กำหนดลำดับของพหุนาม (เช่น degree = 2 คือกำลังสอง)
     - learning_rate : อัตราการเรียนรู้ของ Gradient Descent
     - n_iterations : จำนวนรอบในการปรับค่าน้ำหนัก
   - การสร้าง Polynomial Features
     - ฟังก์ชัน _create_polynomial_features() ทำหน้าที่:
       - เพิ่ม Bias Term (ค่า 1)
       - สร้างพจน์กำลังต่าง ๆ ของข้อมูล
   - Loss Function
     - ใช้ Mean Squared Error (MSE) เป็นฟังก์ชันสูญเสีย
     - เก็บค่า loss ของแต่ละรอบไว้ใน loss_history
     - เก็บค่า validate ของแต่ละรอบไว้ใน val_history
   - ฟังก์ชันประเมินผล
     - mse() : คำนวณค่า Mean Squared Error
     - r2_score() : คำนวณค่า R^2 Score

2. โหลดข้อมูล

   ทำการเรียกใช้ฟังก์ชัน load_regression_data() เพื่อดึงค่า

   - X (Features)
   - y (Target: price ที่ทำ Floor 25)
   - encoder (สำหรับแปลงข้อมูลเชิงหมวดหมู่)

3. แบ่งข้อมูล Train / Test

   ใช้ train_test_split() แบ่งข้อมูลเป็น

   - 80% สำหรับฝึกโมเดล (Training Set)
   - 20% สำหรับทดสอบโมเดล (Test Set)

   โดยกำหนด random_state=42 เพื่อให้ได้ผลลัพธ์ที่สามารถทำซ้ำได้

4. ทำ Feature Scaling

   เนื่องจาก Polynomial Regression ที่ใช้ Gradient Descent มีความไวต่อสเกลของข้อมูล

   จึงทำการ Standardization และเก็บค่า mean และ std ไว้ เพื่อใช้กับข้อมูลใหม่ในอนาคต

5. ฝึกโมเดล

   กำหนดพารามิเตอร์

   - learning_rate = 0.01
   - n_iterations = 2000
   - Degree = 2

   จากนั้นเรียก model.fit() เพื่อเริ่มกระบวนการเรียนรู้

6. การประเมินผลโมเดล

   หลังการฝึก ได้คำนวณค่า

   - Train MSE
   - Test MSE
   - Train R^2
   - Test R^2
   - Training Loss
   - Validation Loss

   เพื่อใช้วัดความแม่นยำของโมเดล และเปรียบเทียบกับโมเดลอื่น

7. การบันทึกโมเดล

   เพื่อให้สามารถนำโมเดลไปใช้งานภายหลังได้ มีการจัดเก็บข้อมูลสำคัญลงในไฟล์ poly_regression_scratch.pkl โดยภายในไฟล์จะประกอบด้วย

   - model (โมเดลที่ฝึกเสร็จแล้ว)
   - mean (ค่าเฉลี่ยสำหรับ scaling)
   - std (ค่าส่วนเบี่ยงเบนมาตรฐานสำหรับ scaling)
   - encoder (ตัวแปลงข้อมูลเชิงหมวดหมู่)

   การจัดเก็บในรูปแบบ pickle ช่วยให้สามารถโหลดโมเดลกลับมาใช้งานสำหรับการทำนาย (Prediction) ได้ทันที โดยไม่ต้องฝึกใหม่

การเลือกฝึกโมเดล Polynomial Regression (From Scikit-learn)

1. โครงสร้างของโมเดล
   - องค์ประกอบหลัก
     - PolynomialFeatures
       - สร้างพจน์กำลังต่าง ๆ ของตัวแปรต้น
       - รวม Interaction term อัตโนมัติ
     - LinearRegression
       - ใช้วิธี Closed-form solution (Ordinary Least Squares)
       - ไม่ใช้ Gradient Descent

        - คำนวณคำตอบได้โดยตรงผ่านสมการเชิงเส้น
    - ฟังก์ชัน
        - fit() สำหรับฝึกโมเดล
        - predict() สำหรับทำนายค่า (รองรับทั้งข้อมูล 1 มิติ และหลายมิติ)
        - mse() สำหรับคำนวณค่า Mean Squared Error
        - r2_score() สำหรับคำนวณค่า Coefficient of Determination (R^2)

2. โหลดข้อมูล

   ทำการเรียกใช้ฟังก์ชัน load_regression_data() เพื่อดึงค่า

   - X (Features)
   - y (Target: price ที่ทำ Floor 25)
   - encoder (สำหรับแปลงข้อมูลเชิงหมวดหมู่)

3. แบ่งข้อมูล Train / Test

   ใช้ train_test_split() แบ่งข้อมูลเป็น

   - 80% สำหรับฝึกโมเดล (Training Set)
   - 20% สำหรับทดสอบโมเดล (Test Set)

   โดยกำหนด random_state=42 เพื่อให้ได้ผลลัพธ์ที่สามารถทำซ้ำได้

4. ทำ Feature Scaling

   เนื่องจาก Polynomial Regression ที่ใช้ Gradient Descent มีความไวต่อสเกลของข้อมูล จึงทำการ Standardization และเก็บค่า mean และ std ไว้ เพื่อใช้กับข้อมูลใหม่ในอนาคต

5. ฝึกโมเดล

   สร้างโมเดล model = PolynomialRegressionSklearn(degree=2) จากนั้นเรียก model.fit() เพื่อเริ่มกระบวนการเรียนรู้ความสัมพันธ์ระหว่างตัวแปรต้นกับตัวแปรตาม

6. การประเมินผลโมเดล

   หลังการฝึก ได้คำนวณค่า

   - Train MSE
   - Test MSE
   - Train R^2
   - Test R^2

   เพื่อใช้วัดความแม่นยำของโมเดล และเปรียบเทียบกับโมเดลอื่น

7. การบันทึกโมเดล

   เพื่อให้สามารถนำโมเดลไปใช้งานภายหลังได้ มีการจัดเก็บข้อมูลสำคัญลงในไฟล์ poly_regression_sklearn.pkl โดยภายในไฟล์จะประกอบด้วย

   - model (โมเดลที่ฝึกเสร็จแล้ว)
   - mean (ค่าเฉลี่ยสำหรับ scaling)
   - std (ค่าส่วนเบี่ยงเบนมาตรฐานสำหรับ scaling)
   - encoder (ตัวแปลงข้อมูลเชิงหมวดหมู่)

การจัดเก็บในรูปแบบ pickle ช่วยให้สามารถโหลดโมเดลกลับมาใช้งานสำหรับการทำนาย (Prediction) ได้ทันที โดยไม่ต้องฝึกใหม่

การเลือกฝึกโมเดล GradientBoostingRegression (From Scratch)

1. โครงสร้างของโมเดล

   โมเดลที่ใช้คือ Gradient Boosting Regression ซึ่งเป็นเทคนิค Ensemble Learning ที่ใช้ Decision Tree ขนาดเล็กหลายต้นมาฝึกแบบต่อเนื่องเพื่อปรับปรุงความแม่นยำของการทำนาย

   องค์ประกอบหลักของโมเดลประกอบด้วย

   - Initial Prediction ใช้ค่าเฉลี่ยของค่า Target (y) เป็นค่าทำนายเริ่มต้น
   - Weak Learners ใช้ Decision Tree Regressor (From Scratch) เป็นตัวเรียนรู้ย่อย
   - Boosting Process ในแต่ละรอบจะเรียนรู้จาก Residual (ค่าความผิดพลาด) ของโมเดลก่อนหน้า
   - Hyperparameters
     - n_estimators = จำนวนต้นไม้ที่ใช้ในโมเดล
     - learning_rate = อัตราการปรับค่าทำนายจากต้นไม้แต่ละต้น
     - max_depth = ความลึกสูงสุดของ Decision Tree

   ฟังก์ชันสำคัญของโมเดล

   - fit() ใช้สำหรับฝึกโมเดล
   - predict() ใช้สำหรับทำนายค่า
   - mse() ใช้คำนวณค่า Mean Squared Error
   - r2_score() ใช้วัดประสิทธิภาพของโมเดล

2. โหลดข้อมูล

   ทำการเรียกใช้ฟังก์ชัน load_regression_data() เพื่อดึงค่า

   - X (Features)
   - y (Target: price ที่ทำ Floor 25)
   - encoder (สำหรับแปลงข้อมูลเชิงหมวดหมู่)

3. แบ่งข้อมูล Train / Test

   ใช้ train_test_split() แบ่งข้อมูลเป็น

   - 80% สำหรับฝึกโมเดล (Training Set)
   - 20% สำหรับทดสอบโมเดล (Test Set)

   โดยกำหนด random_state=42 เพื่อให้ได้ผลลัพธ์ที่สามารถทำซ้ำได้

4. ทำ Feature Scaling

   เนื่องจาก Gradient Boosting Regression ที่ใช้ Gradient Descent มีความไวต่อสเกลของข้อมูล จึงทำการ Standardization และเก็บค่า mean และ std ไว้ เพื่อใช้กับข้อมูลใหม่ในอนาคต

5. ฝึกโมเดล

   กำหนดพารามิเตอร์

   - n_estimators = 200
   - learning_rate = 0.05

- max_depth = 3

จากนั้นเรียก model.fit() เพื่อเริ่มกระบวนการเรียนรู้

6. การประเมินผลโมเดล

   หลังการฝึก ได้คำนวณค่า

   - Train MSE
   - Test MSE
   - Train R^2
   - Test R^2
   - Training Loss
   - Validation Loss

   เพื่อใช้วัดความแม่นยำของโมเดล และเปรียบเทียบกับโมเดลอื่น

7. การบันทึกโมเดล

   เพื่อให้สามารถนำโมเดลไปใช้งานภายหลังได้ มีการจัดเก็บข้อมูลสำคัญลงในไฟล์ gradient_boosting_regression.pkl โดยภายในไฟล์จะประกอบด้วย

   - model (โมเดล Gradient Boosting ที่ฝึกเสร็จแล้ว)
   - mean (ค่าเฉลี่ยของ feature)
   - std (ส่วนเบี่ยงเบนมาตรฐานของ feature)
   - encoder (ตัวแปลงข้อมูลเชิงหมวดหมู่)

   การจัดเก็บในรูปแบบ pickle ช่วยให้สามารถโหลดโมเดลกลับมาใช้งานสำหรับการทำนาย (Prediction) ได้ทันที โดยไม่ต้องฝึกใหม่

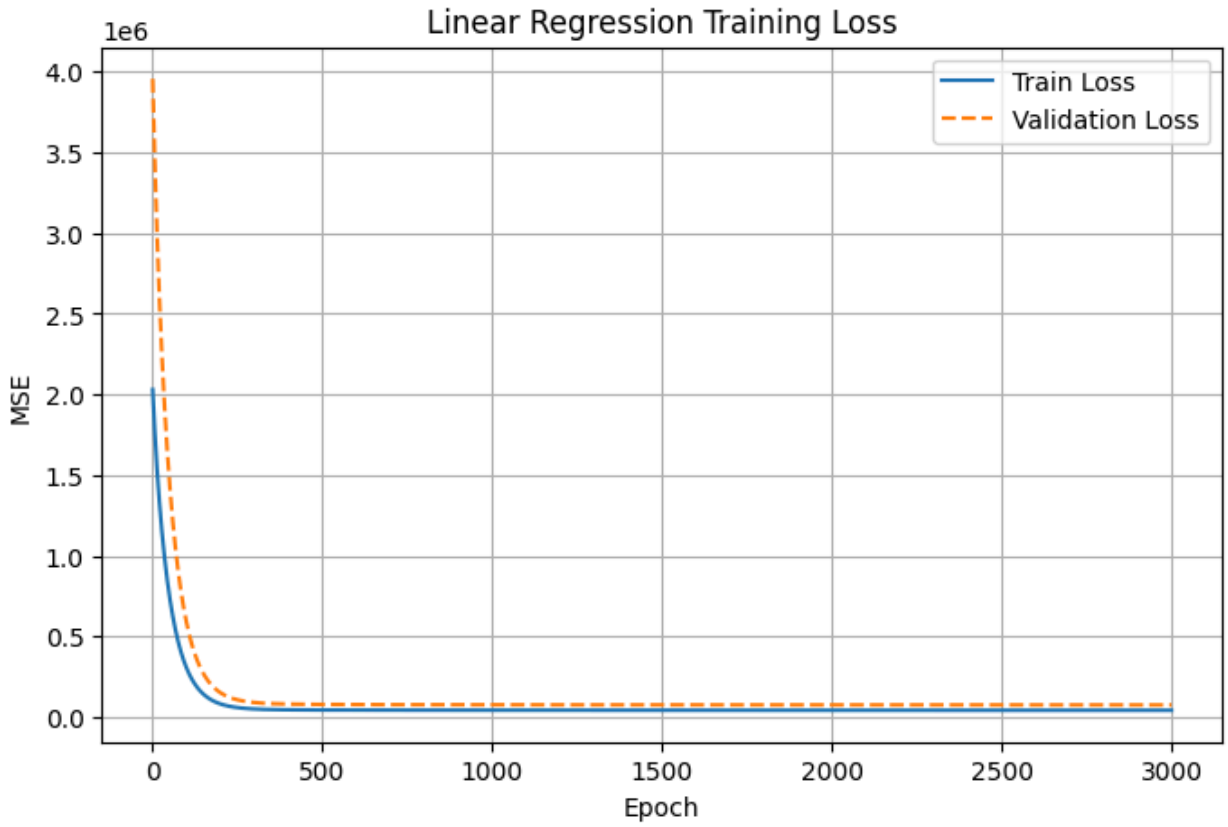
การประเมินโมเดล Regression

การฝึกโมเดลเพื่อดึงค่า Metrics

ทำการเรียกใช้ฟังก์ชัน train() ของแต่ละโมเดล เพื่อให้ได้ค่า Metrics สำหรับเปรียบเทียบ โมเดลที่นำมาทดสอบมีทั้งหมด 7 แบบ ได้แก่

- Linear Regression (Scratch)
- Linear Regression (Scikit-learn)
- Polynomial Regression (Scratch)
- Polynomial Regression (Scikit-learn)
- Multiple Regression (Scratch)
- Multiple Regression (Scikit-learn)
- Gradient Boosting Regression (Scratch)

การประเมินค่า Loss ของโมเดล Regression Scratch



1. การประเมินค่า Loss ของโมเดล Linear Regression Scratch

   จากกราฟ Training Loss และ Validation Loss ของโมเดล Linear Regression แสดงการเปลี่ยนแปลงของค่า Mean Squared Error (MSE) ตามจำนวนรอบการฝึก (Epoch)

   ในช่วงเริ่มต้นของการฝึก โมเดลมีค่า Loss ค่อนข้างสูง โดย

   - Train Loss เริ่มต้นประมาณ 2.0×10^6
   - Validation Loss เริ่มต้นประมาณ 4.0×10^6

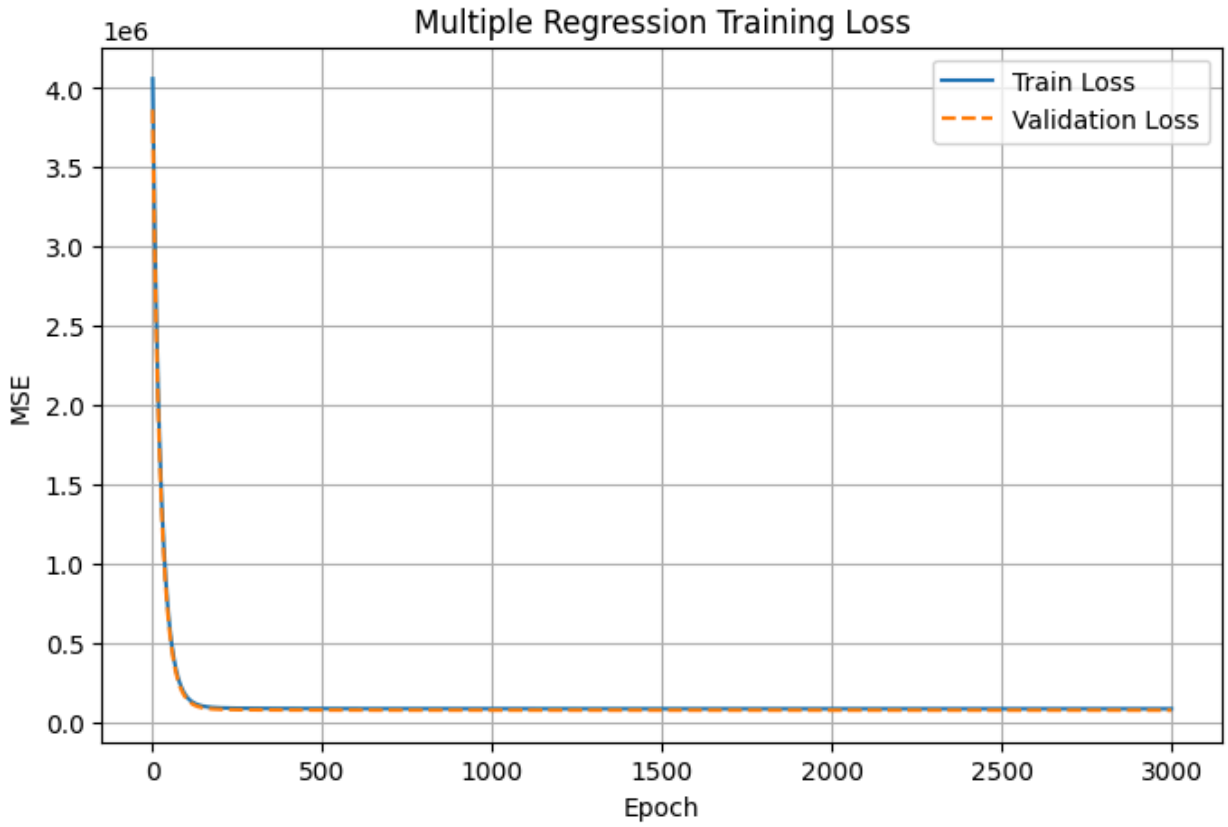
   ซึ่งแสดงว่าในช่วงแรกโมเดลยังไม่สามารถเรียนรู้ความสัมพันธ์ของข้อมูลได้ดี ทำให้ค่าความคลาดเคลื่อนระหว่างค่าที่ทำนายและค่าจริงยังสูง

   เมื่อจำนวน Epoch เพิ่มขึ้น ค่า Loss ของทั้ง Training และ Validation ลดลงอย่างรวดเร็วในช่วงแรกของการฝึก แสดงว่าโมเดลสามารถเรียนรู้รูปแบบของข้อมูลได้ดีขึ้น

   หลังจากประมาณ 200–300 Epoch ค่า Loss เริ่มลดลงช้าลงและเข้าสู่สภาวะคงที่ (Convergence) โดยมีค่า Loss ประมาณ

   - Train Loss $\approx 0.1 \times 10^6$
   - Validation Loss $\approx 0.1 \times 10^6$

   นอกจากนี้ยังพบว่า Training Loss มีค่าต่ำกว่า Validation Loss เล็กน้อยตลอดการฝึก ซึ่งเป็นลักษณะปกติของโมเดล เนื่องจากโมเดลถูกปรับพารามิเตอร์ให้เหมาะกับข้อมูลฝึก (Training Data) จากกราฟยังพบว่าเส้น Training และ Validation Loss มีแนวโน้มใกล้เคียงกันและไม่เกิดการแยกออกจากกันอย่างชัดเจน จึงสามารถสรุปได้ว่า โมเดลไม่มีอาการ Overfitting อย่างรุนแรง และสามารถเรียนรู้จากข้อมูลได้อย่างมีประสิทธิภาพ โดยรวมแล้วโมเดล Linear Regression สามารถลดค่า Loss ลงได้อย่างต่อเนื่องจนเข้าสู่สภาวะเสถียร แสดงให้เห็นว่ากระบวนการฝึกโมเดลมีความเหมาะสมและสามารถนำไปใช้ทำนายค่าได้อย่างมีประสิทธิภาพ



2. การประเมินค่า Loss ของโมเดล Multiple Regression Scratch

จากกราฟ Multiple Regression Training Loss แสดงการเปลี่ยนแปลงของค่า Mean Squared Error (MSE) ของทั้ง Training Loss และ Validation Loss ตามจำนวนรอบการฝึก (Epoch)

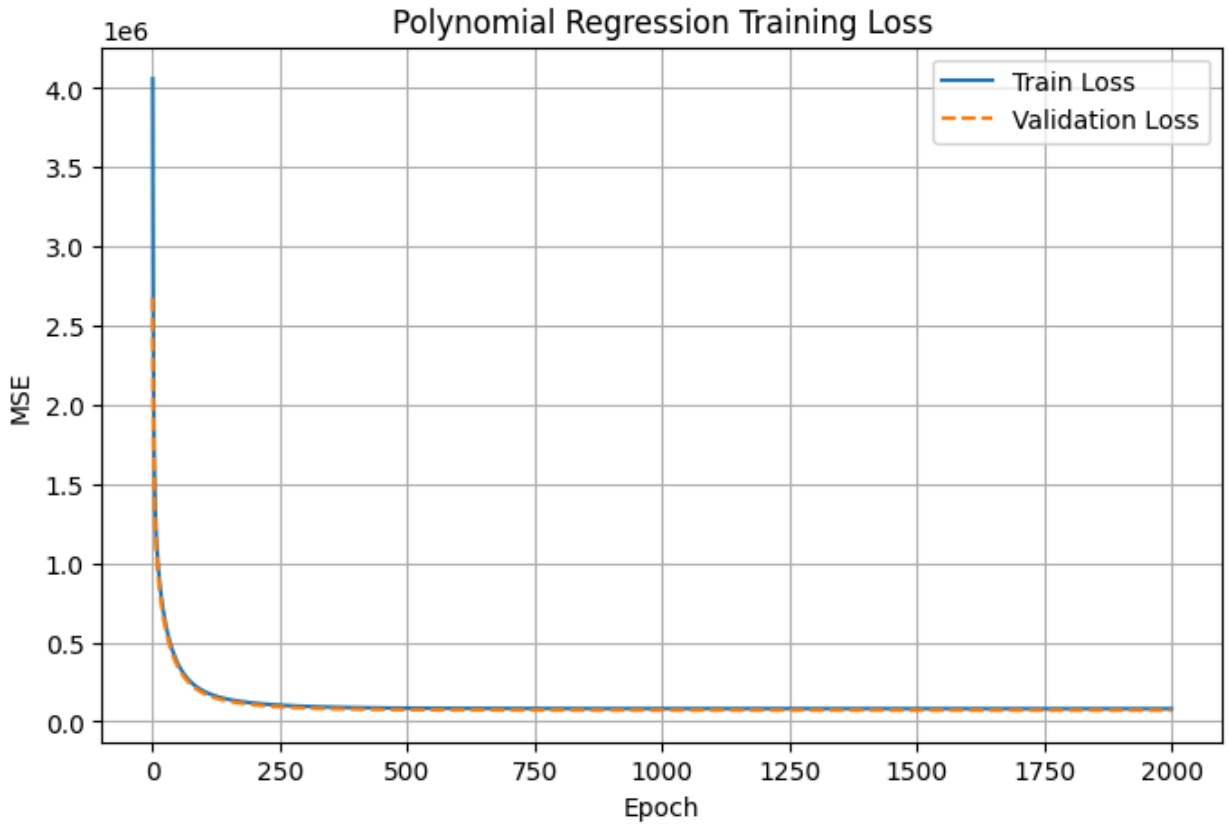
ในช่วงเริ่มต้นของการฝึก โมเดลมีค่า Loss ค่อนข้างสูง โดยมีค่า

- Train Loss เริ่มต้นประมาณ 4.1×10^6
- Validation Loss เริ่มต้นประมาณ 3.9×10^6

ค่า Loss ที่สูงในช่วงแรกแสดงว่าโมเดลยังไม่สามารถเรียนรู้ความสัมพันธ์ระหว่างตัวแปรคุณลักษณะ (features) และราคาของคอมพิวเตอร์ได้อย่างถูกต้อง เมื่อจำนวน Epoch เพิ่มขึ้น ค่า Loss ของทั้ง Training และ Validation ลดลงอย่างรวดเร็วในช่วงแรกของการฝึก แสดงให้เห็นว่าโมเดลสามารถปรับค่าพารามิเตอร์เพื่อเรียนรู้ความสัมพันธ์ของข้อมูลได้ดีขึ้น โดย Multiple Regression ใช้ตัวแปรหลายตัวร่วมกันในการทำนาย ทำให้สามารถอธิบายความสัมพันธ์ของข้อมูลได้ดีกว่า Linear Regression แบบตัวแปรเดียว หลังจากประมาณ 200–300 Epoch ค่า Loss เริ่มลดลงช้าลงและเข้าสู่สภาวะคงที่ โดยมีค่า Loss ประมาณ

- Train Loss $\approx 0.1 \times 10^6$
- Validation Loss $\approx 0.1 \times 10^6$

จากกราฟจะเห็นว่าเส้น Training Loss และ Validation Loss มีแนวโน้มใกล้เคียงกันตลอดการฝึก และไม่มีการแยกออกจากกันอย่างชัดเจน ซึ่งแสดงให้เห็นว่าโมเดล ไม่มีอาการ Overfitting อย่างมีนัยสำคัญ และสามารถเรียนรู้รูปแบบของข้อมูลได้อย่างสมดุล โดยสรุป โมเดล Multiple Regression สามารถลดค่า Loss ได้อย่างรวดเร็วในช่วงต้นของการฝึก และเข้าสู่สภาวะเสถียรเมื่อจำนวน Epoch เพิ่มขึ้น แสดงให้เห็นว่าโมเดลสามารถเรียนรู้ความสัมพันธ์ของหลายคุณลักษณะร่วมกันเพื่อทำนายราคาคอมพิวเตอร์ได้อย่างมีประสิทธิภาพ



3. การประเมินค่า Loss ของโมเดล Polynomial Regression Scratch

จากกราฟ Polynomial Regression Training Loss แสดงการเปลี่ยนแปลงของค่า Mean Squared Error (MSE) ของทั้ง Training Loss และ Validation Loss ตามจำนวนรอบการฝึก (Epoch)

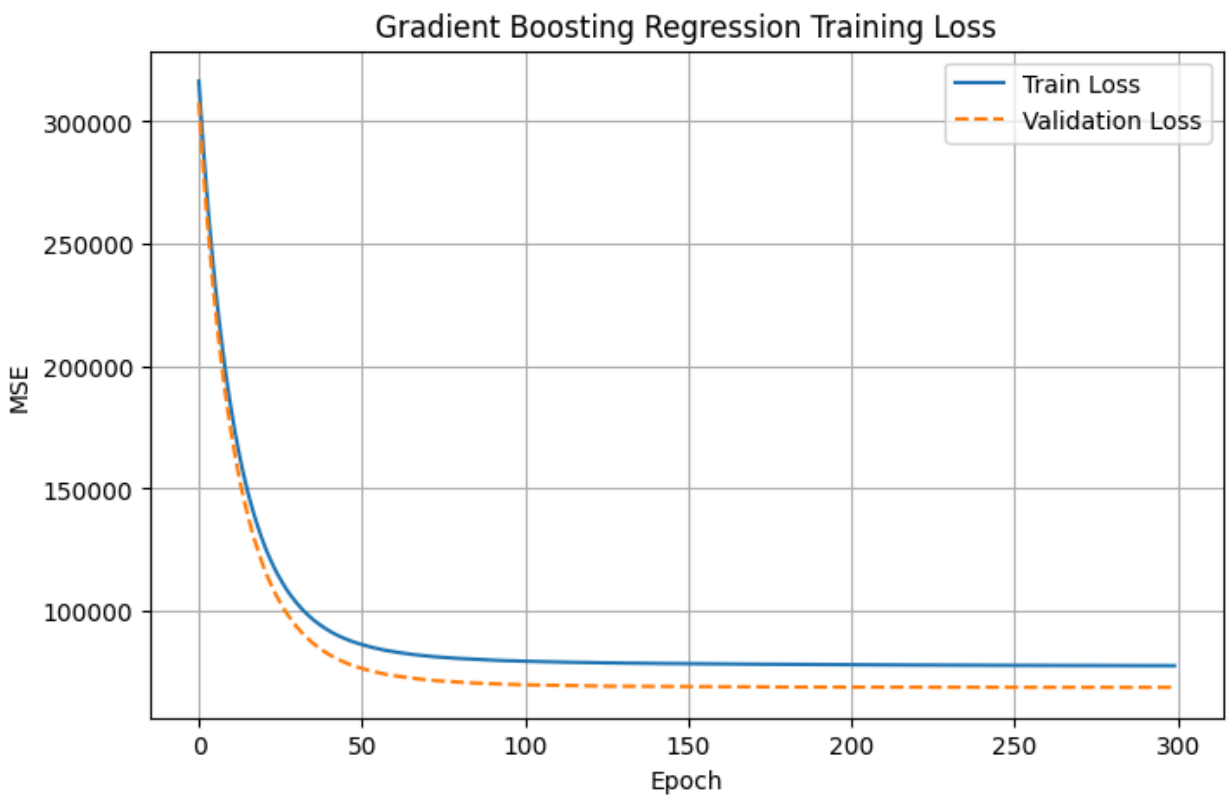
ในช่วงเริ่มต้นของการฝึก โมเดลมีค่า Loss ค่อนข้างสูง โดย

- Train Loss เริ่มต้นประมาณ 4.0×10^6
- Validation Loss เริ่มต้นประมาณ 2.7×10^6

ค่า Loss ที่สูงในช่วงแรกแสดงให้เห็นว่าโมเดลยังไม่สามารถเรียนรู้ความสัมพันธ์ของข้อมูลได้อย่างเหมาะสม เมื่อจำนวน Epoch เพิ่มขึ้น ค่า Loss ของทั้ง Training และ Validation ลดลงอย่างรวดเร็วในช่วงแรกของการฝึก ซึ่งบ่งบอกว่าโมเดลสามารถปรับพารามิเตอร์เพื่อเรียนรู้ความสัมพันธ์แบบไม่เชิงเส้นของข้อมูลได้ดีขึ้น เนื่องจาก Polynomial Regression สามารถจับความสัมพันธ์ที่ซับซ้อนของข้อมูลได้มากกว่า Linear Regression หลังจากประมาณ 200–300 Epoch ค่า Loss เริ่มลดลงช้าลงและเข้าสู่สภาวะคงที่ โดยมีค่า Loss ประมาณ

- Train Loss $\approx 0.1 \times 10^6$
- Validation Loss $\approx 0.1 \times 10^6$

จากกราฟพบว่าเส้น Training Loss และ Validation Loss มีแนวโน้มใกล้เคียงกันตลอดการฝึก และไม่มีการแยกออกจากกันอย่างชัดเจน ซึ่งแสดงให้เห็นว่าโมเดล ไม่มีอาการ Overfitting อย่างรุนแรง และสามารถเรียนรู้รูปแบบของข้อมูลได้อย่างสมดุล โดยรวมแล้วโมเดล Polynomial Regression สามารถลดค่า Loss ลงได้อย่างรวดเร็วในช่วงต้นของการฝึก และเข้าสู่สภาวะเสถียรเมื่อจำนวน Epoch เพิ่มขึ้น แสดงให้เห็นว่าโมเดลสามารถเรียนรู้ความสัมพันธ์เชิงไม่เชิงเส้นของข้อมูลได้อย่างมีประสิทธิภาพ



4. การประเมินค่า Loss ของโมเดล Gradient Boosting Regression Scratch

จากกราฟ Gradient Boosting Regression Training Loss แสดงการเปลี่ยนแปลงของค่า Mean Squared Error (MSE) ของทั้ง Training Loss และ Validation Loss ตามจำนวนรอบการฝึก (Epoch)

ในช่วงเริ่มต้นของการฝึก โมเดลมีค่า Loss ค่อนข้างสูง โดยมีค่า

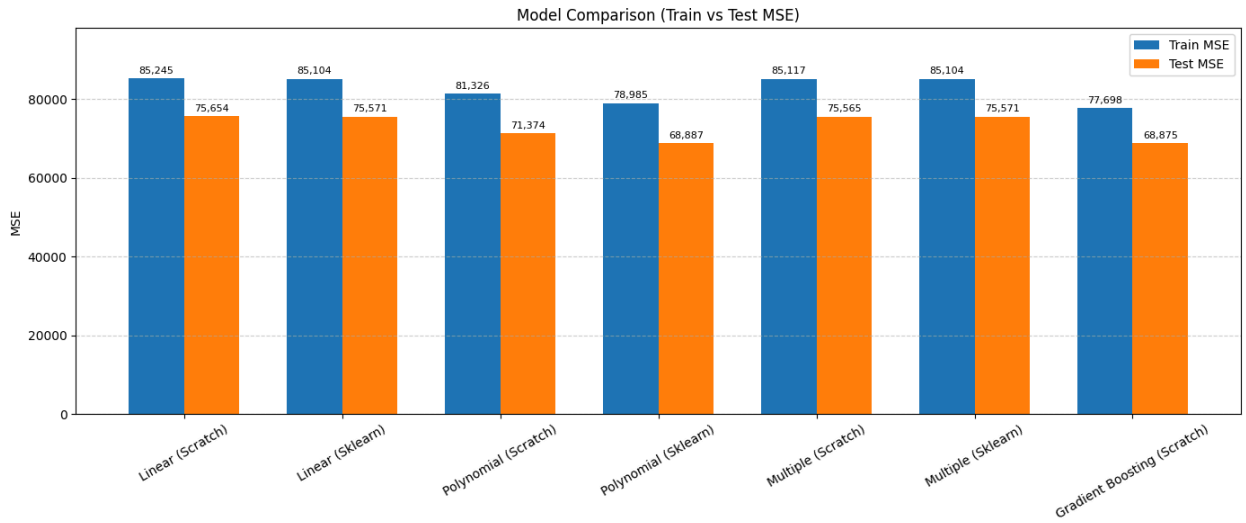
- Train Loss เริ่มต้นประมาณ 315,000
- Validation Loss เริ่มต้นประมาณ 305,000

ค่า Loss ที่สูงในช่วงแรกแสดงว่าโมเดลยังไม่สามารถทำนายค่าราคาได้อย่างแม่นยำ เนื่องจากโมเดลยังไม่ได้เรียนรู้ความสัมพันธ์ของข้อมูล เมื่อจำนวน Epoch เพิ่มขึ้น ค่า Loss ของทั้ง Training และ Validation ลดลงอย่างรวดเร็วในช่วงแรกของการฝึก โดยเฉพาะในช่วง 0–50 Epoch ซึ่งแสดงให้เห็นว่า Gradient Boosting สามารถเรียนรู้รูปแบบของข้อมูลได้อย่างรวดเร็ว โดยการสร้าง Decision Tree หลายต้นต่อเนื่องกัน เพื่อแก้ไขค่าความผิดพลาด (residuals) จากโมเดลก่อนหน้า หลังจากประมาณ 100 Epoch เป็นต้นไป ค่า Loss เริ่มลดลงช้าลง และเข้าสู่สภาวะคงที่ โดยมีค่าประมาณ

- Train Loss ≈ 78,000
- Validation Loss ≈ 70,000

จากกราฟจะเห็นว่า Validation Loss มีค่าต่ำกว่า Train Loss เล็กน้อยและมีแนวโน้มลดลงอย่างต่อเนื่อง แสดงว่าโมเดลสามารถเรียนรู้รูปแบบของข้อมูลได้ดี และไม่มีสัญญาณของ Overfitting อย่างชัดเจน โดยสรุป โมเดล Gradient Boosting Regression สามารถลดค่า Loss ได้อย่างมีประสิทธิภาพตั้งแต่ช่วงต้นของการฝึก และเข้าสู่สภาวะเสถียรเมื่อจำนวน Epoch เพิ่มขึ้น แสดงให้เห็นว่าโมเดลสามารถใช้เทคนิคการรวมหลายต้นไม้ (ensemble) เพื่อปรับปรุงความแม่นยำในการทำนายราคาของข้อมูลได้อย่างมีประสิทธิภาพ

การประเมินค่า MSE ของโมเดล Regression



1. ภาพรวมทั้งหมด
    - ทุกโมเดลมีค่า Test MSE ต่ำกว่า Train MSE
    - ไม่มีโมเดลใดที่ Test MSE สูงกว่า Train MSE อย่างผิดปกติ
    - ค่าทั้งหมดอยู่ในช่วงประมาณ 68,000 – 90,000
    - ไม่มี Overfitting รุนแรง
    - โมเดลมีความเสถียร
2. โมเดลที่ให้ผลดีที่สุด (Test MSE ต่ำสุด)

    Gradient Boosting Regression (Scratch) มี Test MSE ต่ำสุด ตามมาด้วย Polynomial (Sklearn) และ Polynomial (Scratch)
3. การเปรียบเทียบ Scratch vs Scikit-learn
    - Linear Regression
        - Sklearn ให้ผลดีกว่า Scratch เล็กน้อย
    - Polynomial Regression
        - Sklearn ให้ผลดีกว่า Scratch เล็กน้อย
    - Multiple Regression
        - ผลใกล้เคียงกันมาก
4. วิเคราะห์ Generalization

    ทุกโมเดลมีลักษณะ TrainMSE > TestMSE

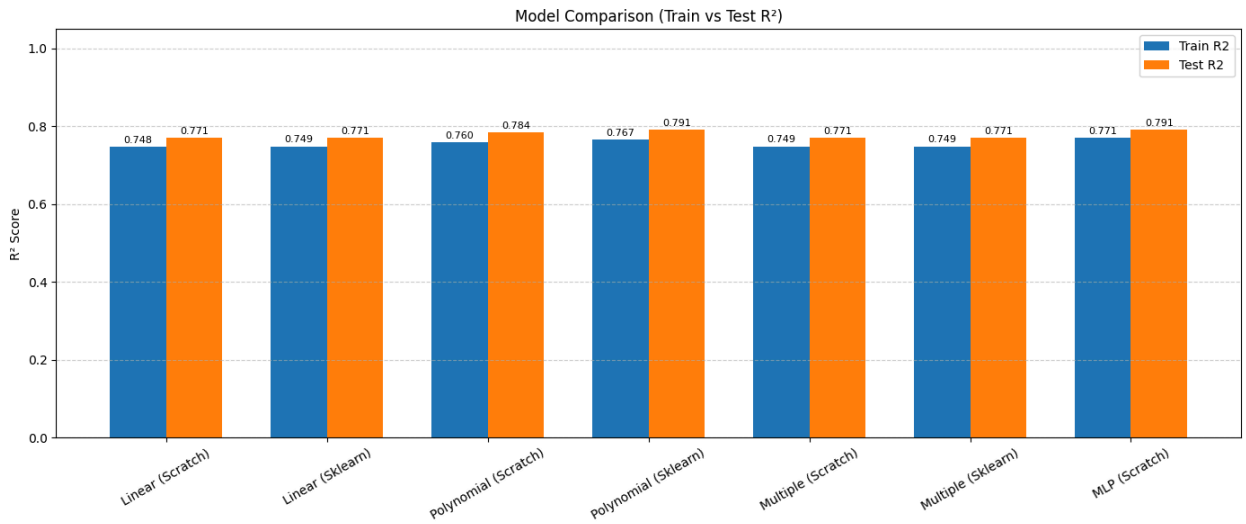
    ซึ่งอาจเกิดจาก
    - Validation/Test set ง่ายกว่า
    - ไม่มี noise มากเท่า Train
    - ไม่มี regularization term ใน Train loss

    ไม่มีโมเดลที่ Train ต่ำมาก แต่ Test สูงมาก แสดงว่าไม่มี Overfitting รุนแรง

5. วิเคราะห์เชิงโครงสร้างโมเดล

โมเดล	ความซับซ้อน	ผลลัพธ์
Linear Regression	ต่ำ	ทำได้ดีระดับพื้นฐาน
Polynomial Regression	ปานกลาง	ดีขึ้นเล็กน้อย
Multiple Regression	ปานกลาง	ใกล้เคียง Linear
Gradient Boosting Regression	สูง	ดีที่สุด

การประเมินค่า R2 ของโมเดล Regression



1. ภาพรวมผลลัพธ์
   - ค่าทั้งหมดอยู่ในช่วงประมาณ 0.73–0.80
   - ทุกโมเดลสามารถอธิบายความแปรปรวนของราคาได้ประมาณ 73% – 80%
   - ไม่มีโมเดลใดที่ R^2 ต่ำกว่า 0.7
   - ถือว่าอยู่ในระดับดี สำหรับปัญหา Regression ทั่วไป
2. โมเดลที่ดีที่สุด (Test R^2 สูงสุด)

   Gradient Boosting Regression (Scratch) และ Polynomial (Sklearn) รองลงมาคือ Polynomial (Scratch)
3. วิเคราะห์ Scratch vs Scikit-learn
   - Linear
     - ค่าค่อนข้างใกล้กันมาก
   - Polynomial
     - Sklearn ให้ผลดีกว่า Scratch เล็กน้อย
   - Multiple

- ค่าค่อนข้างใกล้กันมาก

4. วิเคราะห์ Generalization
   - ทุกโมเดลมี Test R^2 สูงกว่า Train R^2 เล็กน้อย
   - ไม่มี Overfitting
   - โมเดลไม่ได้จำข้อมูล Train มากเกินไป
   - ข้อมูล Test อาจมี pattern ที่สม่ำเสมอกว่า
5. วิเคราะห์เชิงโครงสร้างโมเดล

โมเดล	ความซับซ้อน	Test R^2
Linear	ต่ำ	~0.77
Polynomial	ปานกลาง	~0.80
Multiple	ปานกลาง	~0.77
Gradient Boosting	สูง	~0.80

การฝึกโมเดลเกี่ยวกับ Classification

การเลือกฝึกโมเดล Logistic Regression Classification (From Scratch)

การเลือกฝึกโมเดล Logistic Regression Classification (From Scikit-learn)

การเลือกฝึกโมเดล Decision Tree Classification (From Scratch)

การเลือกฝึกโมเดล Decision Tree Classification (From Scikit-learn)

การเลือกฝึกโมเดล Random Forest Classification (From Scratch)

การเลือกฝึกโมเดล Random Forest Classification (From Scikit-learn)

การเลือกฝึกโมเดล Support Vector Machine Classification (From Scratch)

การเลือกฝึกโมเดล Support Vector Machine Classification (From Scikit-learn)

การเลือกฝึกโมเดล Clustering Classification (From Scratch)

การเลือกฝึกโมเดล Clustering Classification (From Scikit-learn)

การเลือกฝึกโมเดล Perceptron Classification (From Scratch)

การเลือกฝึกโมเดล Perceptron Classification (From Scikit-learn)

การเลือกฝึกโมเดล Multi-layer Perceptron Classification (From Scratch)

การเลือกฝึกโมเดล Multi-layer Perceptron Classification (From Scikit-learn)